

Pure Pure <-> 4TU reconciliation CLI

Overview

This command-line tool compares related-publication metadata between a Pure export CSV and 4TU.ResearchData metadata retrieved through the 4TU API.

It is designed to support review and reconciliation. It does **not** update Pure or 4TU.ResearchData directly. Instead, it creates two CSV reports that help identify where publication links or publication metadata may be missing.

The tool checks two directions:

1. Missing in Pure

Pure has no related research output UUID for a dataset, but 4TU.ResearchData already contains publication-related metadata.

2. Missing in 4TU

Pure has a related research output UUID for a dataset, but 4TU.ResearchData does not contain publication-related metadata.

What the tool is for

Use this CLI when you want to:

- compare Pure dataset-publication links with 4TU.ResearchData metadata;
- identify datasets where Pure may be missing a related publication link;
- identify datasets where 4TU may be missing publication metadata;
- limit the check to a specific institutional group, such as Wageningen University and Research;
- optionally include child groups below the selected institutional group;
- create reviewable CSV reports before making any manual corrections in Pure or 4TU.

Reconciliation logic

The script reads a semicolon-separated Pure export CSV and splits records into two candidate groups.

Candidate group 1: `missing_in_pure`

A dataset is added to this group when:

- the row has a value in `UUID 4TU`; and
- the row has no value in `UUID Research output`.

The script then queries the corresponding 4TU dataset record. If 4TU contains at least one publication-related metadata field, the dataset is written to the `missing_in_pure` report.

4TU publication-related metadata is considered present when at least one of these fields is non-empty:

- `resource_title`
- `resource_doi`
- `references`

Candidate group 2: `missing_in_4tu`

A dataset is added to this group when:

- the row has a value in `UUID 4TU`; and
- the row has a value in `UUID Research output`.

The script then queries the corresponding 4TU dataset record. If 4TU has no publication-related metadata, the dataset is written to the `missing_in_4tu` report.

4TU publication-related metadata is considered absent when all of these fields are empty:

- `resource_title`
- `resource_doi`
- `references`

Workflow

The command-line workflow is:

1. Read the Pure export CSV.
2. Validate that all required columns are present.
3. Split rows into two candidate groups:
 - datasets potentially missing a related publication link in Pure;
 - datasets potentially missing publication metadata in 4TU.
4. Retrieve the list of groups from the 4TU `/v3/groups` endpoint.
5. Resolve the configured group name to a numeric 4TU group ID.
6. Optionally collect all descendant child group IDs.
7. Query each candidate dataset using `/v2/articles/<dataset_uuid>`.
8. Keep only datasets that belong to the selected group or allowed child groups.
9. Write two output CSV reports.

Required input CSV columns

The input file must be a semicolon-separated CSV file, usually exported from Pure.

The script expects the following columns:

Column	Purpose
<code>UUID 4TU</code>	Dataset UUID used to query the 4TU API.
<code>Title Dataset</code>	Dataset title from Pure.
<code>DOI Dataset</code>	Dataset DOI from Pure.
<code>UUID Research output</code>	Pure research output UUID. Used to decide whether Pure already has a related publication.
<code>Type Research output</code>	Type of the related Pure research output. Used in the <code>missing_in_4tu</code> report.
<code>DOI Research output</code>	DOI of the Pure research output. Used in the <code>missing_in_4tu</code> report.
<code>Title Research output</code>	Title of the Pure research output. Used in the <code>missing_in_4tu</code> report.

Rows without a UUID 4TU value are skipped because the script cannot query 4TU without a dataset UUID.

API endpoints used

The script uses two 4TU.ResearchData API endpoints.

Endpoint	Purpose
/v3/groups	Retrieves available groups so the script can resolve the selected institutional group name to a numeric group ID.
/v2/articles/<dataset_uuid>	Retrieves metadata for each dataset UUID found in the Pure export.

By default, the API base URL is:

`https://data.4tu.nl`

Installation requirements

The script requires Python 3 and the `requests` package.

Install the dependency with:

```
pip install requests
```

Basic usage

Run the script with the path to the Pure export CSV:

```
python wur_pure_4tu_reconcile.py pure_export.csv
```

This creates two output files using the default names:

```
wur_datasets_missing_pure_related_publication.csv
```

```
wur_datasets_missing_4tu_related_publication.csv
```

Command-line options

Positional argument

Argument	Required	Description
<code>input_csv</code>	Yes	Path to the semicolon-separated Pure export CSV.

Optional arguments

Option	Default	Description
<code>--output-csv-missing-in-pure</code>	<code>wur_datasets_missing_pure_reconciled_publications.csv</code>	Output CSV file name where Pure may be missing a publication link.
<code>--output-csv-missing-in-4tu</code>	<code>wur_datasets_missing_4tu_reconciled_publications.csv</code>	Output CSV file name where 4TU may be missing publication metadata.
<code>--base-url</code>	<code>https://data.4tu.nl</code>	Base URL of the 4TU.ResearchData instance.
<code>--group-name</code>	Wageningen University and Research	Exact group name as used in the 4TU /v3/groups API.
<code>--include-descendants</code>	Disabled	Also include child groups below the selected group.
<code>--sleep</code>	0.2	Pause between API requests, in seconds.
<code>--timeout</code>	30	HTTP timeout, in seconds.
<code>--verbose</code>	Disabled	Print progress information to standard error.

Example commands

1. Run with all defaults

```
python reconcile_related_publications.py pure_export.csv
```

Use this when:

- your input file is called `pure_export.csv`;
- you want to check the default WUR group;
- you are happy with the default output file names.

2. Choose custom output file names

```
python reconcile_related_publications.py pure_export.csv \
  --output-csv-missing-in-pure review_missing_in_pure.csv \
  --output-csv-missing-in-4tu review_missing_in_4tu.csv
```

Use this when you want report names that are easier to share or archive.

3. Include child groups

```
python reconcile_related_publications.py pure_export.csv \
  --include-descendants
```

Use this when datasets may belong to subgroups of the selected institution rather than directly to the parent group.

4. Use a different group name

```
python reconcile_related_publications.py pure_export.csv \
  --group-name "Delft University of Technology"
```

Use this when you want to run the same reconciliation workflow for a different institutional group. The group name must match the group name exposed by the 4TU /v3/groups API.

5. Use a different 4TU API base URL

```
python reconcile_related_publications.py pure_export.csv \  
  --base-url "https://data.4tu.nl"
```

This is useful if you are testing against another 4TU.ResearchData instance or a staging environment.

6. Increase the request timeout

```
python reconcile_related_publications.py pure_export.csv \  
  --timeout 60
```

Use this if API requests are slow or if you are working with a large input file.

7. Slow down API requests

```
python reconcile_related_publications.py pure_export.csv \  
  --sleep 1.0
```

Use this to be more conservative with API traffic.

8. Print progress information

```
python reconcile_related_publications.py pure_export.csv \  
  --verbose
```

This prints information such as the resolved group IDs and the dataset currently being processed.

Output report 1: missing in Pure

The `missing_in_pure` report contains datasets where:

- Pure has no related research output UUID; and
- 4TU contains at least one publication-related metadata field.

Default output file:

`wur_datasets_missing_pure_related_publication.csv`

Columns:

Column	Description
UUID Dataset	4TU dataset UUID.
Title Dataset	Dataset title from Pure, or from 4TU if missing in Pure.
DOI Dataset	Dataset DOI from Pure, or from 4TU if missing in Pure.
UUID Research output	Empty in this report, because Pure has no linked research output UUID.

Column	Description
<code>group_id</code>	Group ID returned by 4TU.
<code>4TU_resource_title</code>	Publication title stored in 4TU.
<code>4TU_resource_doi</code>	Publication DOI stored in 4TU.
<code>4TU_references</code>	References field returned by 4TU.

Typical interpretation:

4TU seems to know about a related publication, but Pure does not have the corresponding related research output UUID linked to the dataset.

Output report 2: missing in 4TU

The `missing_in_4tu` report contains datasets where:

- Pure has a related research output UUID; and
- 4TU has no publication-related metadata.

Default output file:

`wur_datasets_missing_4tu_related_publication.csv`

Columns:

Column	Description
UUID Research output	Pure research output UUID.
Type Research output	Type of the Pure research output.
UUID Dataset	4TU dataset UUID.
DOI Research output	DOI of the Pure research output.
Title Research output	Title of the Pure research output.
Title Dataset	Dataset title from Pure, or from 4TU if missing in Pure.
DOI Dataset	Dataset DOI from Pure, or from 4TU if missing in Pure.
<code>group_id</code>	Group ID returned by 4TU.
<code>4TU_resource_title</code>	Publication title stored in 4TU. Expected to be empty for rows in this report.
<code>4TU_resource_doi</code>	Publication DOI stored in 4TU. Expected to be empty for rows in this report.
<code>4TU_references</code>	References field returned by 4TU. Expected to be empty for rows in this report.

Typical interpretation:

Pure knows about a related research output, but the corresponding 4TU dataset record does not contain publication-related metadata.

How group filtering works

The script first retrieves all groups from:

```
/v3/groups
```

It then looks for an exact case-insensitive match using the value passed to:

```
--group-name
```

By default this is:

```
Wageningen University and Research
```

Only datasets belonging to the selected group are included in the reports.

If you use:

```
--include-descendants
```

then the script also includes child groups below the selected root group. This is useful when datasets are assigned to departments, institutes, or subgroups instead of the parent institution.

How publication metadata is detected in 4TU

The script checks these fields in the 4TU article metadata response:

```
resource_title
resource_doi
references
```

For the **references** field, the script accepts multiple possible API shapes:

- a non-empty list;
- a non-empty string;
- a non-empty dictionary;
- another truthy value.

This makes the check more robust if the API response shape varies slightly.

Error handling

The script is designed to continue when an individual dataset cannot be fetched.

For example, if one dataset UUID causes a request error, the script prints a warning and continues with the next dataset.

However, the script stops with an error when:

- the input CSV is missing required columns;
- the selected group name cannot be found in `/v3/groups`;
- the selected group name matches multiple group IDs;
- the `/v3/groups` response cannot be interpreted;
- another unexpected error occurs.

Troubleshooting

Error: input CSV is missing required columns

Check that the input CSV contains all required columns exactly as expected:

```
UUID 4TU
Title Dataset
DOI Dataset
UUID Research output
Type Research output
DOI Research output
Title Research output
```

Column names are matched exactly, including spaces.

Error: could not find group

The value passed to `--group-name` must match a group name in the 4TU /v3/groups API.

Try running the script with the default group name first:

```
python reconcile_related_publications.py pure_export.csv --verbose
```

If the group name is still not found, inspect the available group names in the API response and update `--group-name` accordingly.

The output CSV files are empty

This can be valid. It means no records matched the reconciliation criteria after filtering by group.

Check the following:

- Does the input CSV contain values in UUID 4TU?
- Is the correct group name being used?
- Should `--include-descendants` be enabled?
- Does 4TU actually contain `resource_title`, `resource_doi`, or `references` for the relevant datasets?
- Does Pure actually contain UUID Research output values for the datasets expected to appear in `missing_in_4tu`?

API requests are timing out

Increase the timeout:

```
python reconcile_related_publications.py pure_export.csv --timeout 60
```

You can also slow down requests:

```
python reconcile_related_publications.py pure_export.csv --sleep 1.0
```

Recommended review workflow

1. Run the script with `--verbose` and `--include-descendants` if relevant.
2. Open both generated CSV reports.

3. Review `missing_in_pure` to identify datasets where Pure may need an additional related research output link.
4. Review `missing_in_4tu` to identify datasets where 4TU may need publication metadata added.
5. Validate a sample of rows manually in both systems before making changes.
6. Keep the generated reports as audit/review evidence for the reconciliation process.

Notes for maintainers

The script intentionally separates the two candidate dictionaries:

- `candidates_missing_in_pure`
- `candidates_missing_in_4tu`

It then builds the union of dataset UUIDs and fetches each 4TU dataset only once. This avoids unnecessary API calls while preserving both reconciliation directions.

This is important because the same dataset UUID could theoretically appear in both candidate groups in the input export. Keeping the candidate rows separate prevents one case from overwriting the other.

Summary

This CLI provides a safe, review-first reconciliation workflow between Pure and 4TU.ResearchData. It helps identify:

- records where Pure may be missing a related research output link; and
- records where 4TU may be missing publication-related metadata.

The output is limited to reviewable CSV files, making the script suitable for audit, manual correction workflows, and iterative data quality checks.